Piloto gratuito (dois setores)

Documentação técnica - Prumo

> **Situação:** o código está pronto para este piloto (modo `s3` do armazenamento, testado com um emulador S3 local). As contas e serviços abaixo ainda não foram criados. Os limites gratuitos citados foram conferidos em outubro de 2026 e podem mudar.

1. Arquitetura do piloto

```
Navegador -> Frontend (Cloudflare Pages)
    |
    +-> Backend (Cloud Run) -> PostgreSQL (Neon)
    |         |
    |         +- assina URLs de envio e leitura
    |
    +-> Arquivos (Cloudflare R2, bucket privado)
          envio direto por URL assinada (PUT) e leitura por URL assinada (GET)
```

Peça	Serviço	Plano gratuito (out/2026)
Banco	Neon PostgreSQL	1 GB por projeto, 100 horas de processamento por mês, sem cartão. Desliga após 5 min sem uso e religa na próxima consulta
Arquivos	Cloudflare R2	10 GB por mês, 1 milhão de envios e 10 milhões de leituras por mês, sem cobrança de download
Backend	Google Cloud Run	2 milhões de requisições por mês. Exige conta de faturamento
Frontend	Cloudflare Pages	Site estático

Estimativa para dois setores: 20 manuais x (vídeo de 10 min ~190 MB + PDF) ≈ 4 GB de arquivos; o banco usa poucos MB.

2. Antes de começar (validar)

- **LGPD:** o banco guarda nome e login de colaboradores e o bucket guarda material interno. Confirme com Segurança da Informação a região dos dados (os planos gratuitos costumam ficar nos EUA ou na Europa).
- **Cartão:** Cloud Run (e possivelmente o R2) pedem forma de pagamento mesmo dentro da cota gratuita. Configure alerta de orçamento em R$ 0,01 no Google Cloud.
- **Região do Cloud Run:** confirme que a região escolhida entra na cota gratuita.

3. Banco no Neon

1. Crie um projeto no Neon, na região mais próxima disponível.
2. Copie a string de conexão **direta** (sem `-pooler` no host), com `sslmode=require`.
3. Aplique o schema:

```
$env:DATABASE_URL='postgresql://usuario:senha@ep-xxxx.região.aws.neon.tech/neondb?sslmode=require'
psql "$env:DATABASE_URL" -v ON_ERROR_STOP=1 -f backend/sql/schema.sql
```

4. Entre com `admin` / `admin123` e troque a senha no primeiro acesso.

Use a conexão direta, não a com pooler: o pooler (PgBouncer) pode recusar os parâmetros de tempo limite que o backend envia. Se precisar do pooler, defina `DB_STATEMENT_TIMEOUT_MS=0` e `DB_IDLE_TRANSACTION_TIMEOUT_MS=0`.

4. Bucket no Cloudflare R2

1. Crie o bucket (ex.: `manual-operacional-piloto`) **sem** acesso público e sem domínio público.
2. Em **Manage R2 API Tokens**, crie um token com permissão *Object Read & Write* limitado a esse bucket. Guarde o *Access Key ID* e o *Secret Access Key*.
3. Anote o endpoint S3 da conta: `https://<ID_DA_CONTA>.r2.cloudflarestorage.com`.
4. Configure o CORS do bucket com o endereço do frontend:

```
[
  {
    "AllowedOrigins": ["https://manual-operacional.pages.dev"],
    "AllowedMethods": ["GET", "PUT", "HEAD"],
    "AllowedHeaders": ["Content-Type", "Range"],
    "ExposeHeaders": ["ETag", "Content-Length", "Content-Range", "Accept-Ranges"],
    "MaxAgeSeconds": 3600
  }
]
```

Sem o CORS, o envio na tela Criar Manual falha com "Falha de rede ao enviar o arquivo" e o PDF não abre.

5. Backend no Cloud Run

Variáveis (segredos no Secret Manager, não em texto):

Variável	Valor no piloto
`DATABASE_URL`	String direta do Neon (segredo)
`JWT_SECRET`	Texto aleatório longo (segredo)
`FRONTEND_URL`	Endereço do Cloudflare Pages
`S3_BUCKET`	`manual-operacional-piloto`
`S3_ENDPOINT`	`https://<ID_DA_CONTA>.r2.cloudflarestorage.com`
`S3_REGION`	`auto`
`S3_ACCESS_KEY_ID` / `S3_SECRET_ACCESS_KEY`	Token do R2 (segredos)
`DB_POOL_MAX`	`5`
`DEV_DEMO_MODE`	`false`

```
gcloud run deploy manual-operacional-api \
  --source=backend \
  --region=SUA_REGIAO \
  --allow-unauthenticated \
  --max-instances=2 \
  --set-env-vars="FRONTEND_URL=https://manual-operacional.pages.dev,S3_BUCKET=manual-operacional-piloto,S3_ENDPOINT=https://ID_DA_CONTA.r2.cloudflarestorage.com,S3_REGION=auto,DB_POOL_MAX=5,DEV_DEMO_MODE=false" \
  --set-secrets="DATABASE_URL=database-url:latest,JWT_SECRET=jwt-secret:latest,S3_ACCESS_KEY_ID=r2-key-id:latest,S3_SECRET_ACCESS_KEY=r2-secret:latest"
```

- `--max-instances=2` com `DB_POOL_MAX=5` limita o backend a 10 conexões no Neon.
- Sem instância mínima, o primeiro acesso depois de um tempo parado demora alguns segundos (o banco do Neon também religa). É o custo de ficar no gratuito.
- Health checks: `/health` (vida) e `/health/ready` (consulta o banco).
- Os arquivos não passam pelo Cloud Run, então o limite de 32 MB por requisição não afeta os vídeos.

6. Frontend no Cloudflare Pages

Configuração	Valor
Pasta raiz	`frontend`
Comando de build	`npm run build`
Pasta de saída	`dist`
Variável `VITE_API_URL`	Endereço do Cloud Run

Depois do primeiro deploy, confira se `FRONTEND_URL` no backend e o CORS do bucket usam exatamente o endereço final do Pages.

7. Teste de aceitação do piloto

- [ ] `GET /health/ready` responde `200`.
- [ ] Login com a senha nova do admin.
- [ ] Cadastro dos dois setores, dos gerentes e dos operadores.
- [ ] Criar Manual mostra "Enviando vídeo... N%" e termina com "Manual cadastrado com sucesso".
- [ ] Operador abre o manual: PDF aparece, vídeo toca e permite avançar.
- [ ] Operador de outro setor não vê o manual.
- [ ] Link copiado do PDF para de abrir após 4 horas.
- [ ] Painel de Uso mostra o progresso.

8. Acompanhamento

- **Neon:** uso de armazenamento e horas de processamento no painel do projeto. Ao passar do limite de horas, o banco fica suspenso até o mês seguinte.
- **R2:** armazenamento total no painel do bucket.
- **Cloud Run:** requisições e alerta de orçamento.

9. Migração depois do piloto

O piloto foi montado para trocar de fornecedor sem mudar o código:

- **Banco:** é PostgreSQL padrão. Exporte e importe no novo fornecedor (ex.: Cloud SQL):

```
pg_dump --no-owner --no-acl "$NEON_URL" | psql "$NOVO_BANCO_URL"
```

- **Arquivos:** o banco guarda só as chaves (`pdfs/<uuid>.pdf`, `videos/<uuid>.mp4`). Copie os objetos mantendo as mesmas chaves (ex.: `rclone copy`) e troque as variáveis `S3_*`. Para AWS S3 ou outro serviço compatível com S3, basta isso.
- **Google Cloud Storage:** o GCS aceita a API S3 pela interoperabilidade (chaves HMAC, endpoint `https://storage.googleapis.com`). **Hipótese a validar em homologação:** URLs assinadas geradas pelo SDK da AWS funcionando no GCS. Se não funcionarem, cria-se um módulo GCS com as mesmas funções de `backend/src/config/objectStorage.js`, sem mexer no restante.
- Faça a troca com o sistema em manutenção, confira contagem de linhas e de objetos e só então apague o piloto.